Bash Shell Scripting — One-Page Cheat Sheet

Practical Bash for now: write scripts, take input, use conditions, loops, functions, files, and system commands.

1. Start a Script

Use a shebang. Make the file executable once. Run it directly or with bash.

```
#!/usr/bin/env bash
chmod +x script.sh
./script.sh
bash script.sh
# comment
```

2. Output, Variables, Input

No spaces around =. Quote "$var". read -r is the safe default.

```
name="Raghav"
age=20
echo "Hello $name"
printf "%s\n" "$name"
read -r -p "Enter name: " user
echo "User = $user"
```

3. Command Substitution + Arithmetic

Capture command output with $(...). Use $((...)) or ((...)) for integer math.

```
today=$(date)
host=$(hostname)
me=$(whoami)
sum=$((a + b))
((count++))
```

4. Useful Special Variables

These matter when scripts take arguments or when debugging.

```
$0   # script name
$1   # first argument
$2   # second argument
$#   # number of arguments
$?   # last exit status
$$   # current shell PID
```

5. if / elif / else

Numeric tests: -eq -ne -lt -le -gt -ge. File tests: -f -d -e. String tests often use [[ ]].

```
read -r -p "Age: " age
if [ "$age" -lt 0 ]; then
  echo "Invalid"
elif [ "$age" -lt 18 ]; then
  echo "Minor"
else
  echo "Adult"
fi
```

6. Common Tests

Keep these in memory. They cover most beginner use cases.

```
[ "$a" -eq "$b" ]   # equal
[ "$a" -lt "$b" ]   # less than
[[ "$x" == "yes" ]] # string equal
[[ -z "$name" ]]    # empty string
[ -f "app.log" ]    # file exists
[ -d "data" ]       # directory exists
```

7. Loops

for = known range/list. while = repeat while condition stays true.

```
for ((i=0; i<5; i++)); do
  echo "$i"
done

count=0
while [ "$count" -lt 5 ]; do
  echo "$count"
  ((count++))
done
```

8. break / continue

break exits the loop. continue skips the rest of the current iteration.

```
while true; do
  read -r -p "q to quit: " x
  [[ "$x" == "q" ]] && break
  [[ -z "$x" ]] && continue
  echo "You typed $x"
done
```

9. Functions

Use functions to avoid repeating code. Function arguments are $1, $2, ...

```
greet() {
  echo "Hello $1"
}
greet "Raghav"
```

10. Files + Redirection

> overwrites. >> appends. cat reads. mkdir -p safely creates directories.

```
mkdir -p logs
touch logs/app.log
echo "start" > logs/app.log
echo "next" >> logs/app.log
cat logs/app.log
```

11. System + Process Commands

These are directly useful in scripts and match what you practiced.

```
date          # current date/time
hostname      # machine name
whoami        # current user
uname -a      # OS/kernel info
ps            # processes
ps -p "$$"    # current shell process
```

12. Basic Template

This is enough to build simple scripts on your own.

```
#!/usr/bin/env bash
read -r -p "File: " file
if [ -f "$file" ]; then
  lines=$(wc -l < "$file")
  echo "Lines: $lines"
else
  echo "File not found"
fi
```

Rules: Always test commands in terminal first. Quote variables: "$var". Use indentation. Prefer small scripts + functions.